

Lecture 19:

Inference-time Algorithms

Speaker: Omar Khattab

Outline

1. Inference on LLMs.
2. Reasoning Models.
3. Language Agents.

The Bitter Lesson

Rich Sutton

March 13, 2019

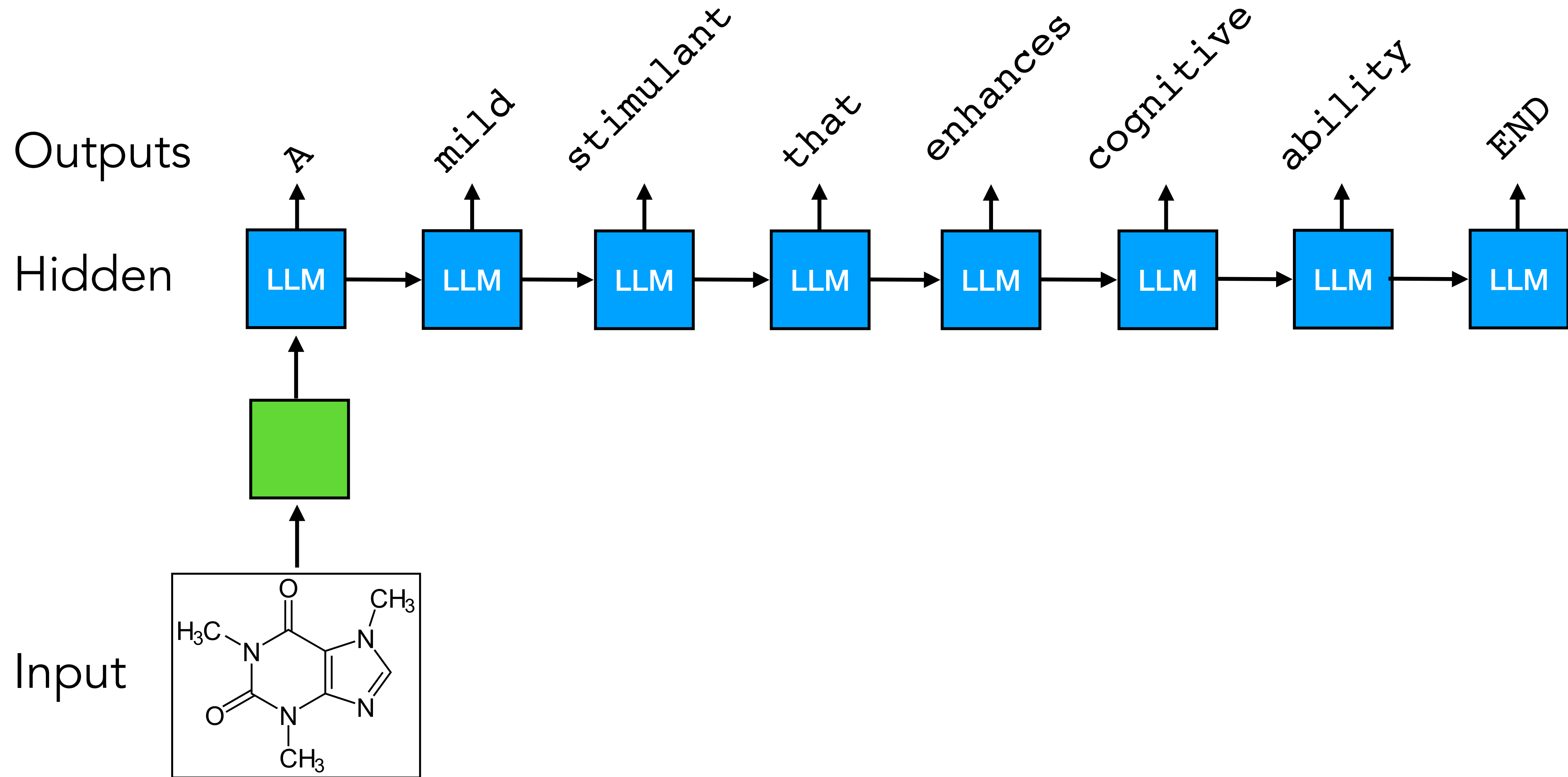
The bitter lesson shows the power of general purpose methods that scale well with increased computation: *search* (“*inference*”) and *learning* (“*training*”).

Recap: Large Language Models

- We reduced many general capabilities to **language modeling**, i.e. being able to assign a probability to a sequence of tokens.
- We decomposed that into modeling **next-token probability**!
We pre- and post-trained models to be quite good at this.
- Next-token modeling is by itself a significant inductive bias.

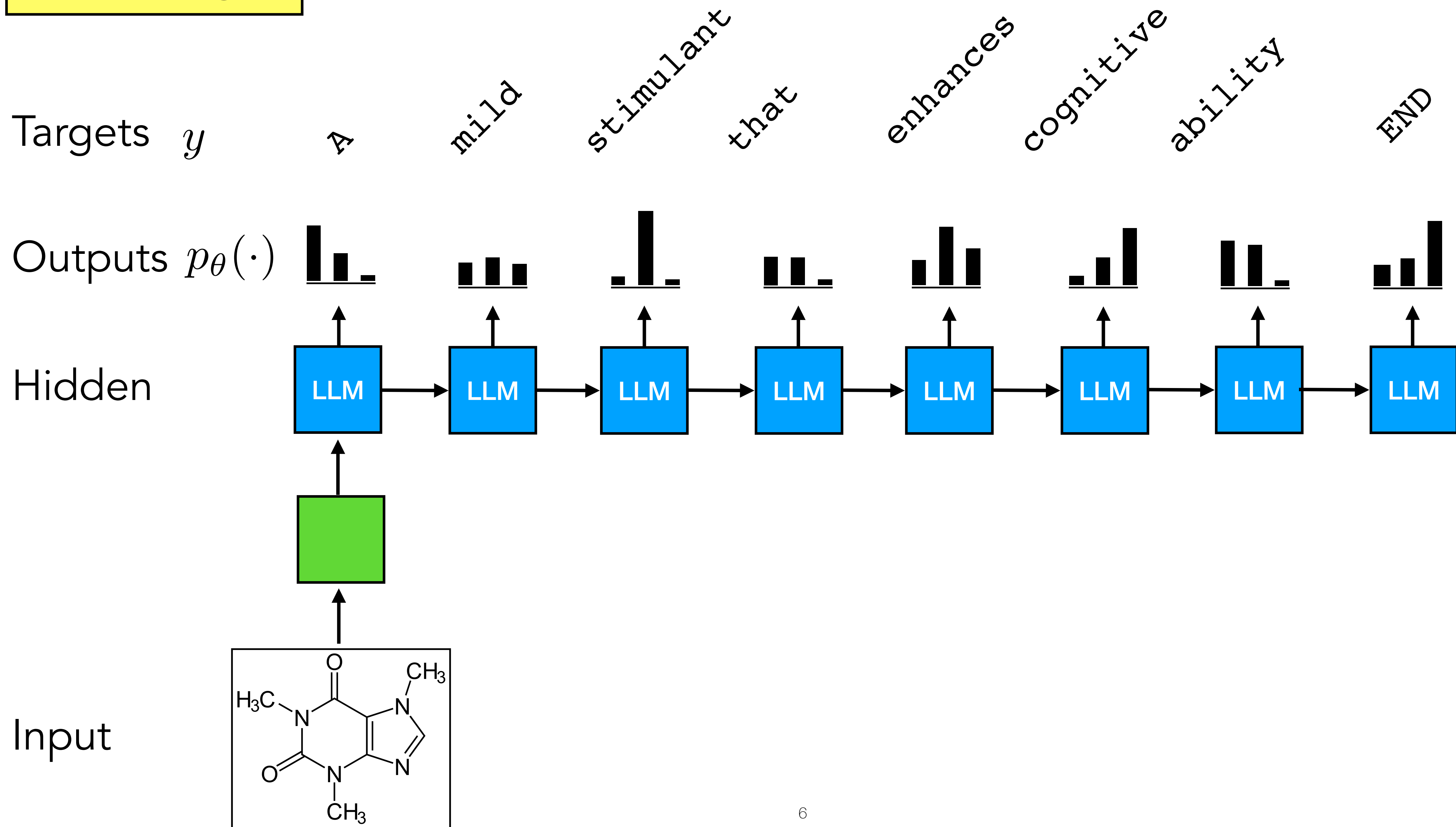
What are some consequences of this design?

Recap: Training vs. inference with an "autoregressive" model



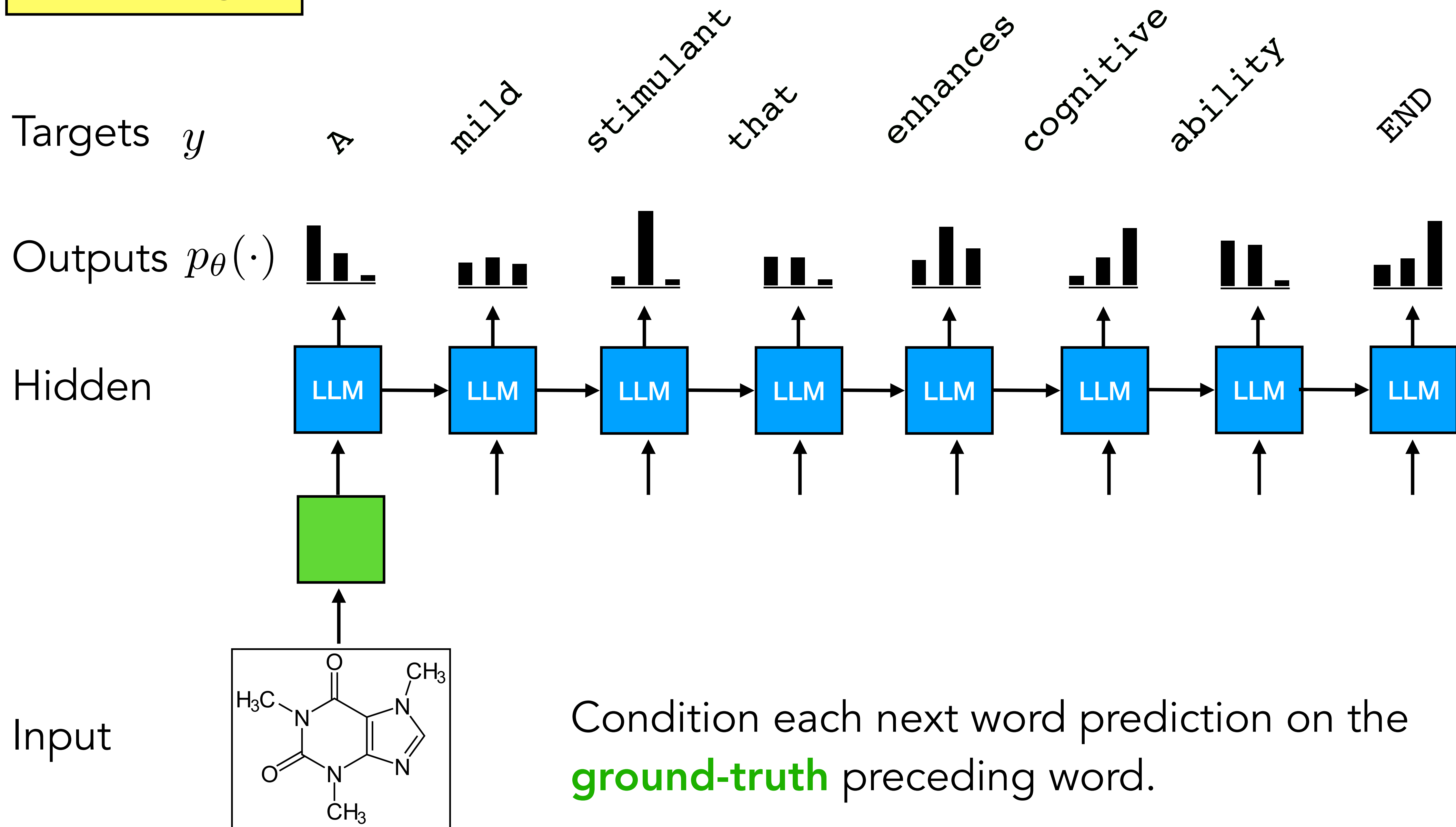
Training

Minimize cross-entropy over a corpus



Training

Implicit here is "teacher forcing"



Inference

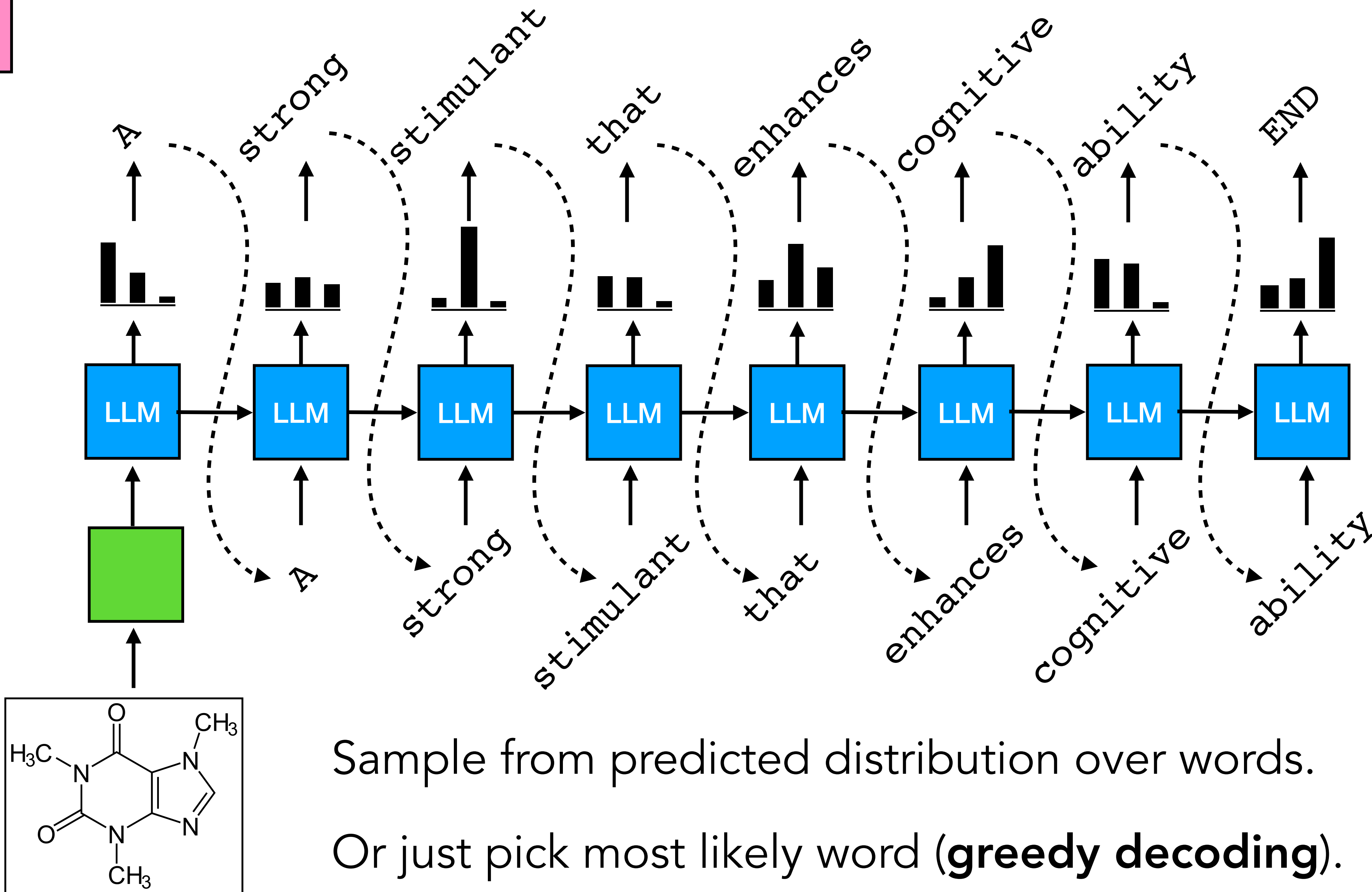
Samples

Outputs $p_{\theta}(\cdot)$

Hidden

Issues?

Input



Does greedy decoding produce the most probable sequence?

$$\begin{array}{c} \{\mathbf{x}_1^*, \dots, \mathbf{x}_n^*\} = \arg \max_{\mathbf{X}} p(\mathbf{X}) \\ \underbrace{\hspace{1.5cm}} \\ p(\mathbf{X}) = \underbrace{p(\mathbf{x}_n | \mathbf{x}_1, \dots, \mathbf{x}_{n-1})}_{\mathbf{x}_n^{*'} = \arg \max_{\mathbf{x}_n}} \cdots \underbrace{p(\mathbf{x}_2 | \mathbf{x}_1)}_{\mathbf{x}_2^{*'} = \arg \max_{\mathbf{x}_2}} \underbrace{p(\mathbf{x}_1)}_{\mathbf{x}_1^{*'} = \arg \max_{\mathbf{x}_1}} \end{array}$$

How could that failure happen?

And how can we tackle it?

Idea: Best-of- N

Sample N sequences iid and pick the **sequence with highest probability**:

$$p_{\theta}(\mathbf{y}_1, \dots, \mathbf{y}_T | \mathbf{x}) = \prod_i p_{\theta}(\mathbf{y}_i | \mathbf{y}_1, \dots, \mathbf{y}_{i-1}, \mathbf{x})$$

A stimulant that enhances cognitive function. $\longrightarrow$ p_{θ} $\longrightarrow$ 0.07

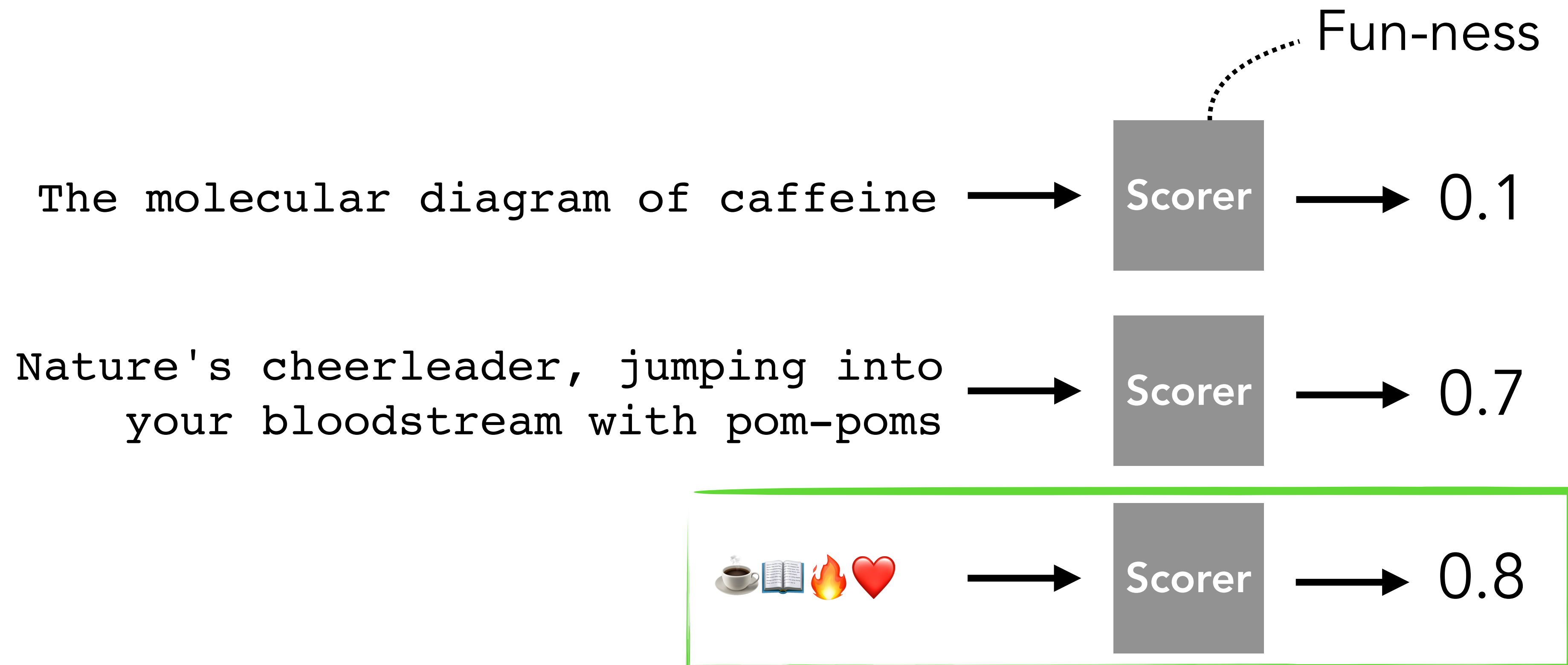
A stimulant that improves thinking. $\longrightarrow$ p_{θ} $\longrightarrow$ 0.05

A chemical that makes you feel more awake. $\longrightarrow$ p_{θ} $\longrightarrow$ 0.08

...

Search Errors vs. Model Errors

- Did we our inference strategy just fail to find the best-scoring output?
- Or does our model assign a higher probability to a “worse” output?



Building scoring functions (recall: IR)
can be really hard. Different approach?

The molecular diagram of caffeine → Scorer → 1.0

Scientific
accuracy

Nature's cheerleader, jumping into
your bloodstream with pom-poms → Scorer → 0.3

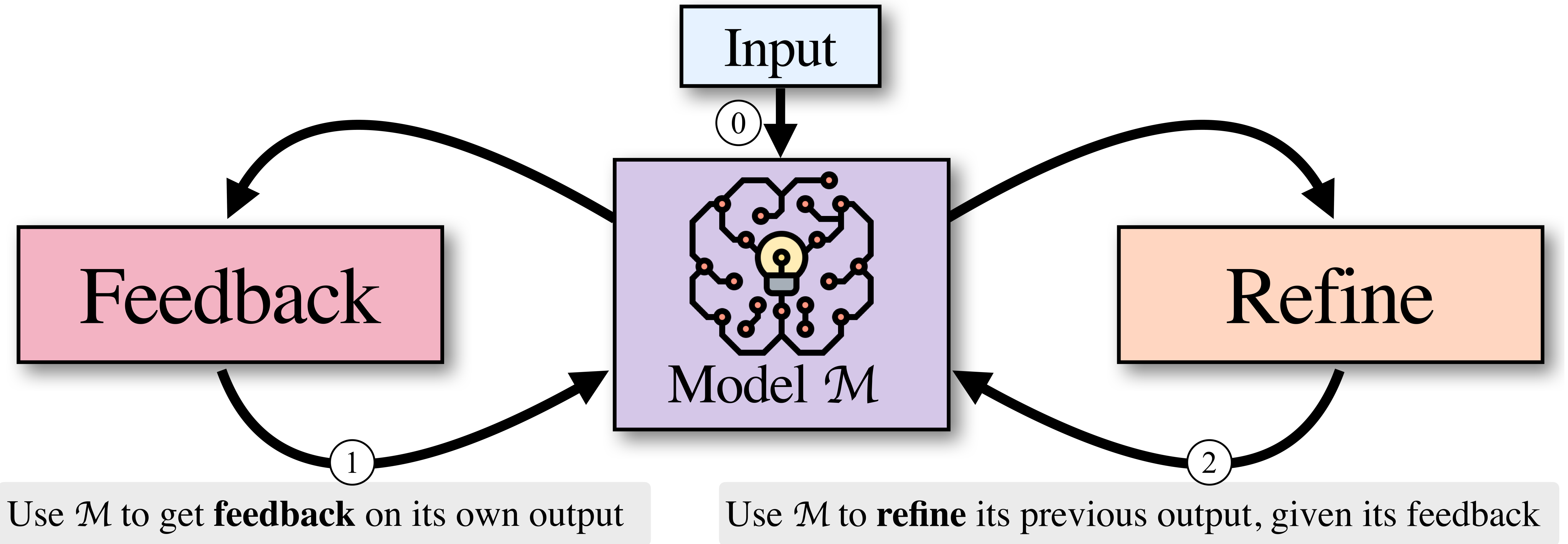


→ Scorer → 0.1

...

Inference

Sequential self-refinement, instead of parallel search?



Intuition: *it can be (much) easier to **iterate** than to produce a perfect first draft.*
But this can introduce errors to correct outputs too!

Idea: **Higher-level inductive biases**

For the most part, we talked about **architectural** inductive biases, like convolution, recurrence, or attention.

We now have strong **priors from pre-training**, e.g. problem solving strategies or knowledge of different skills.

Can we design “**reasoning**” biases?

Reasoning Models

Recall: **Chain-of-Thought** as an emergent capability

If prompted well, even self-supervised LLMs **verbalize their thinking** in a way that leads to much higher accuracy for some problems.

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✓

Fundamental fact about deep learning? Or about language?

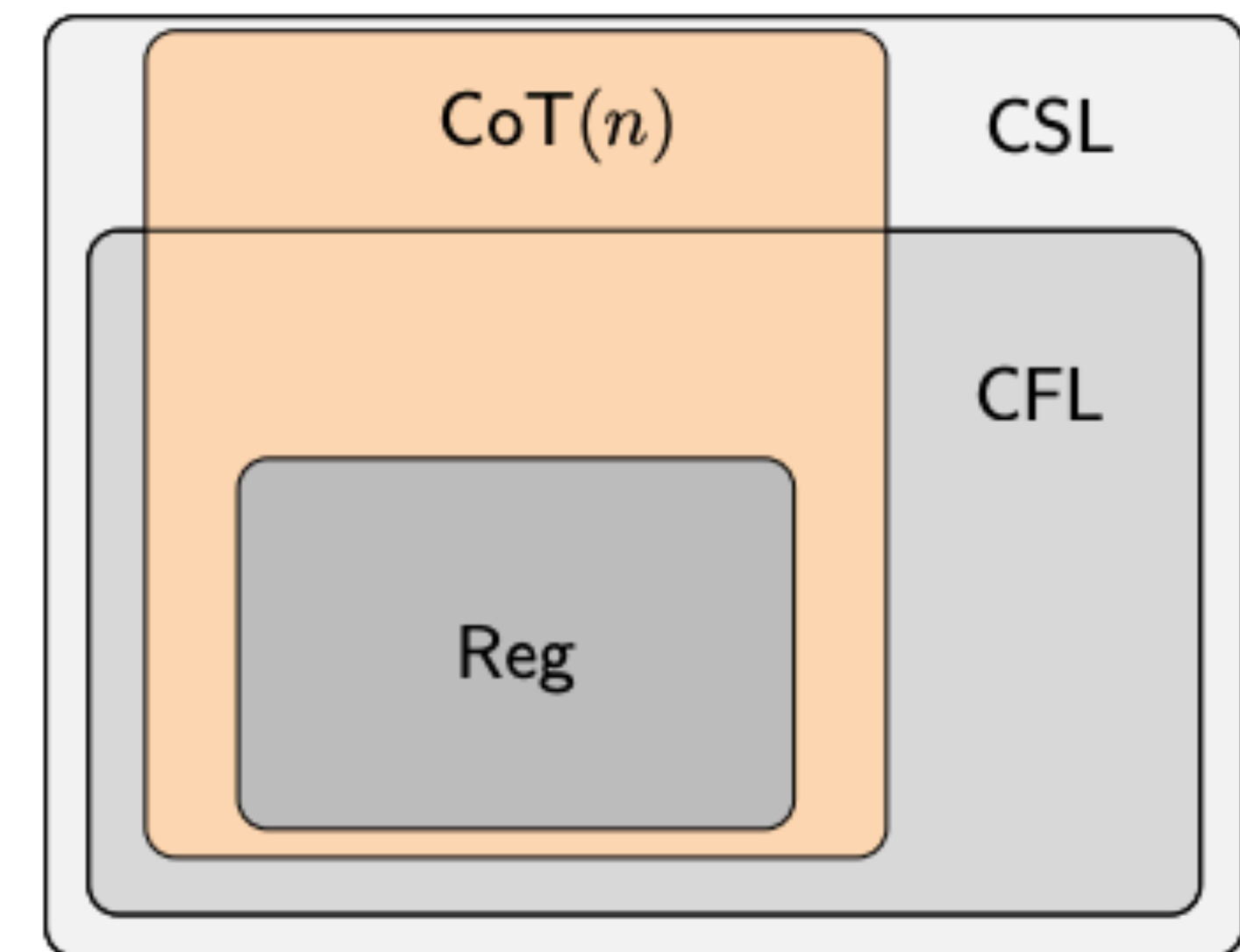
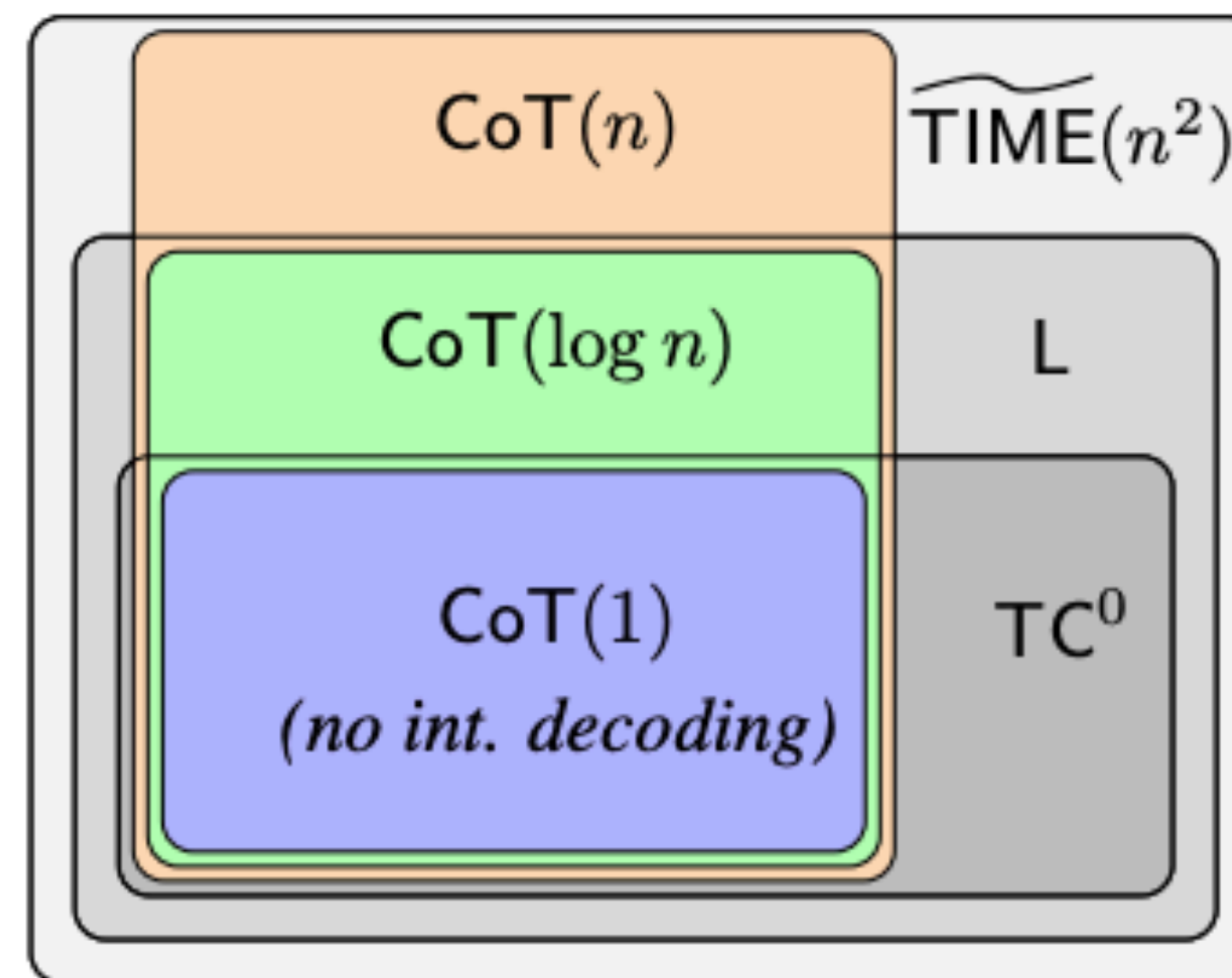
Or a tedious trick to eke out scores on niche benchmarks?

Intuitively, many problems are much easier if you **break them down into steps**. *More steps unlock harder problems!*

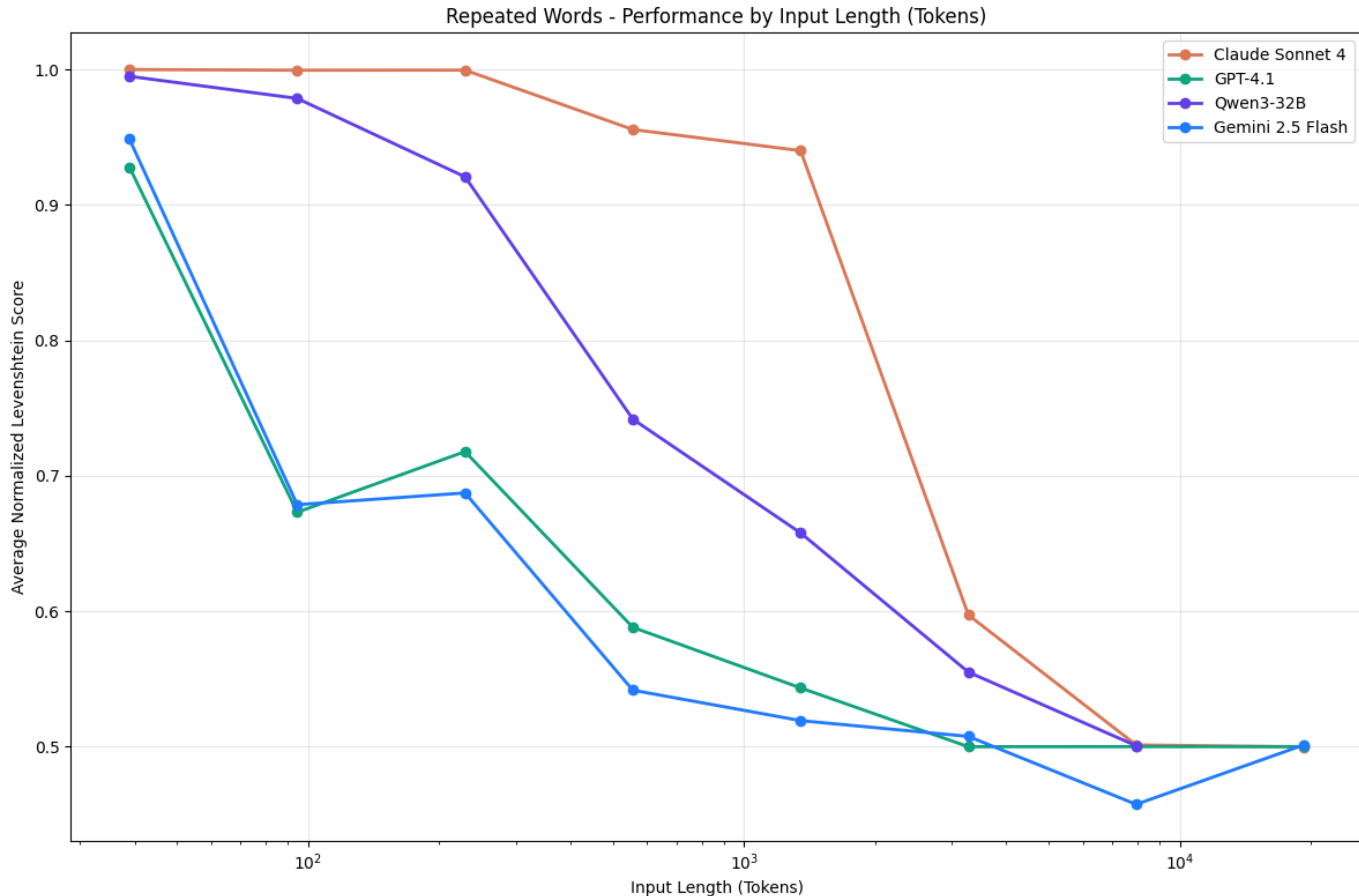
Can we study this formally, as a function of **input size N** and the **allowed “length” of the chain of thought?**

Transformers without a CoT have no choice of weights that can, say, perfectly check if two nodes in a (large enough) graph are connected.

But long enough CoT expands their expressive power a lot!



"Context rot" with longer prompts in practice...



*Context Rot: How Increasing
Input Tokens Impacts LLM
Performance (2025)*

With SFT/RLHF, CoT became an **implicit behavior** that LLMs sometimes leveraged during conversation.

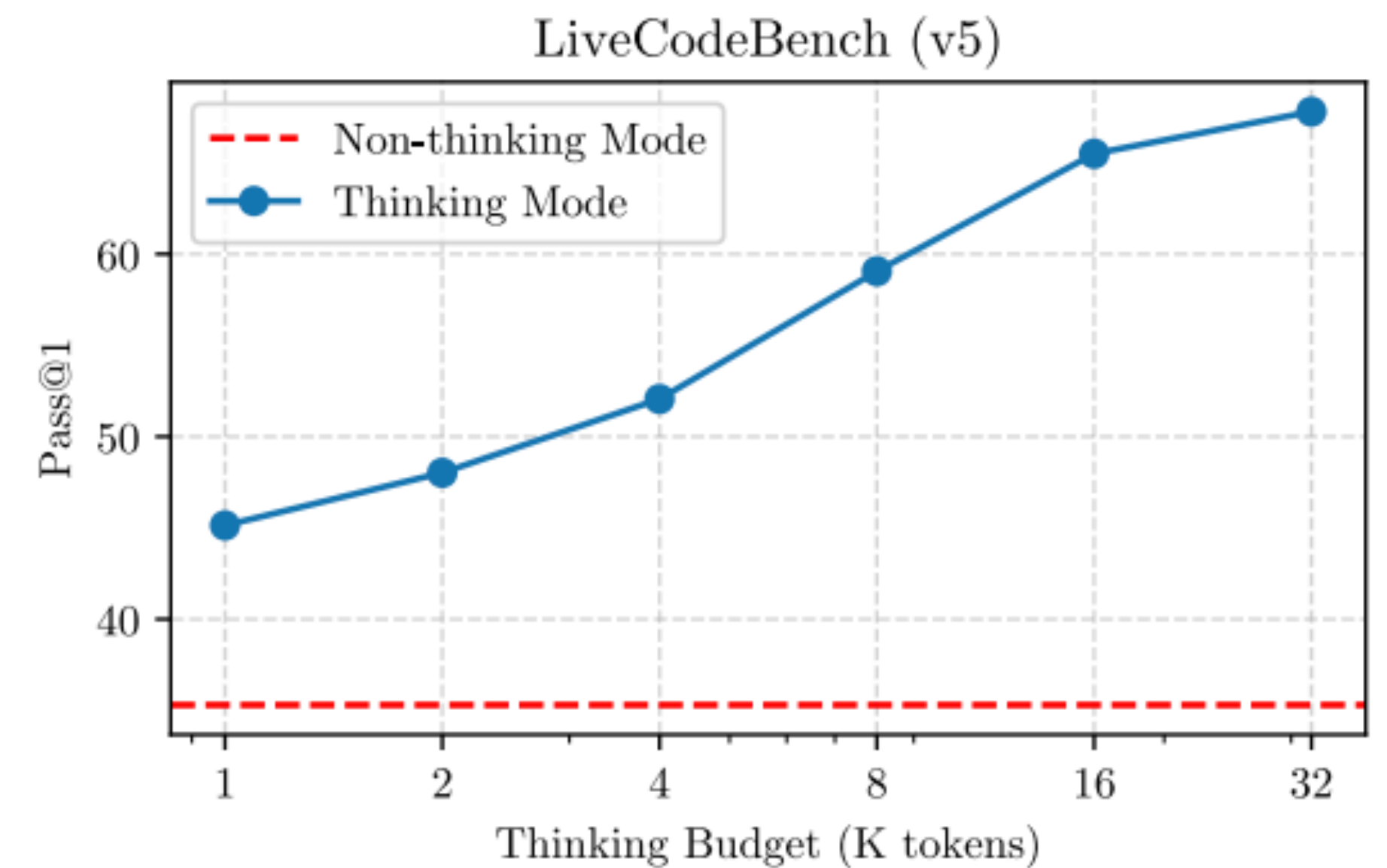
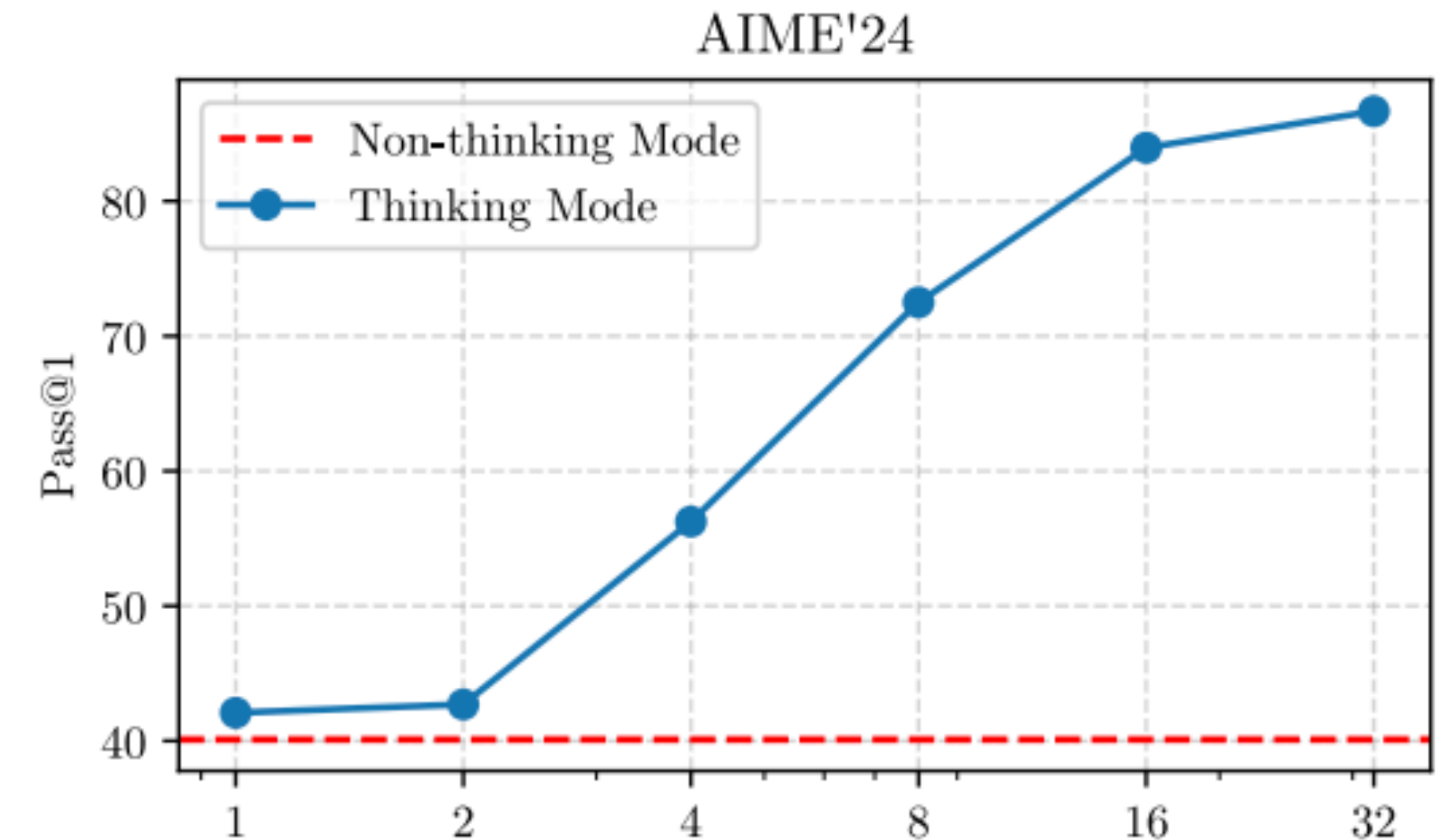
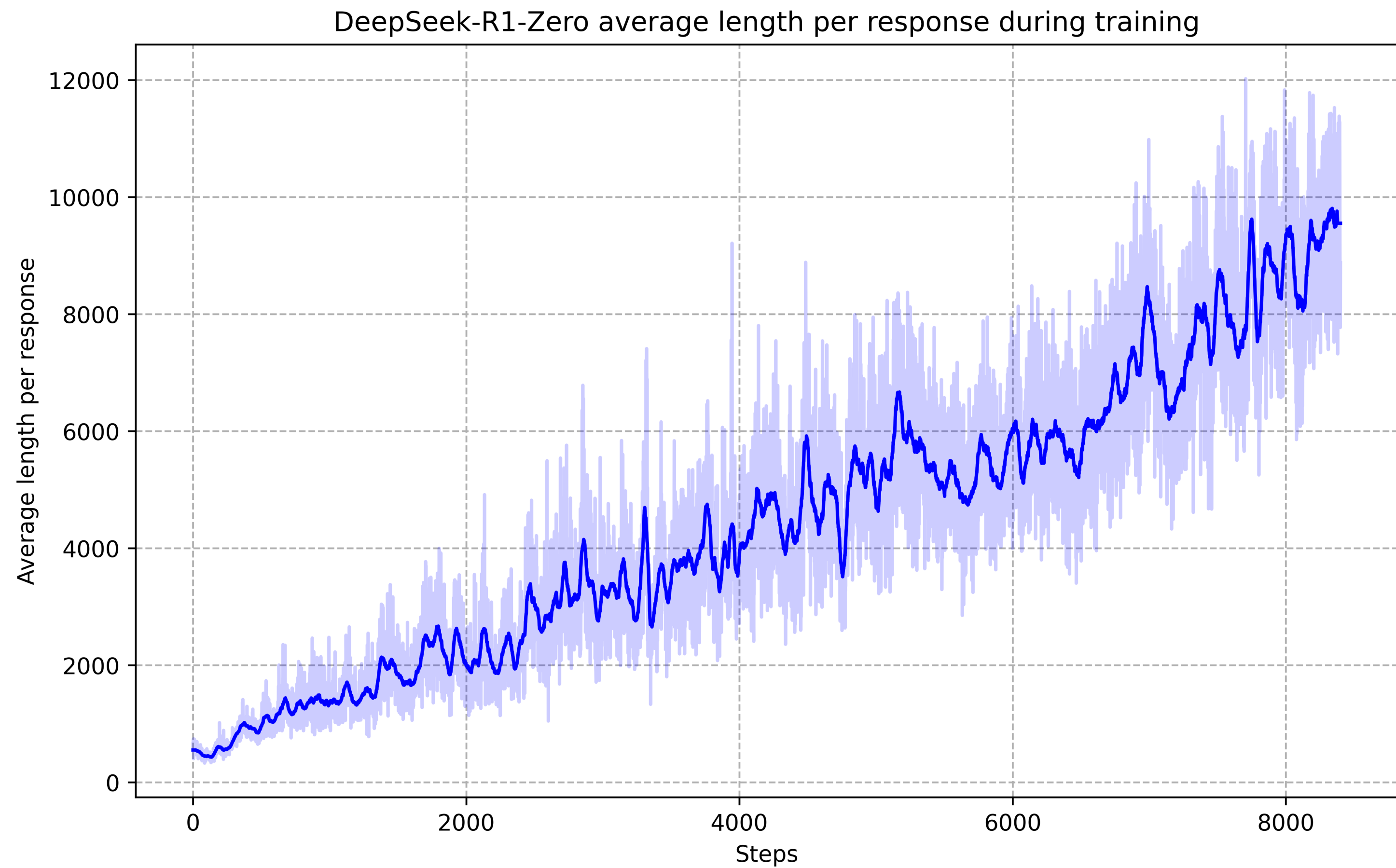
Reasoning models: Inject CoT as **inductive bias** that change expressive power, compared with a Transformer!

To produce response **y** given prompt **x**, *always* sample latent reasoning **z** first. Train the model with RL so it teaches itself what **z** should look like!

Even as our LLM systems get more complex, we can just apply REINFORCE or its variants (e.g., “GRPO”).

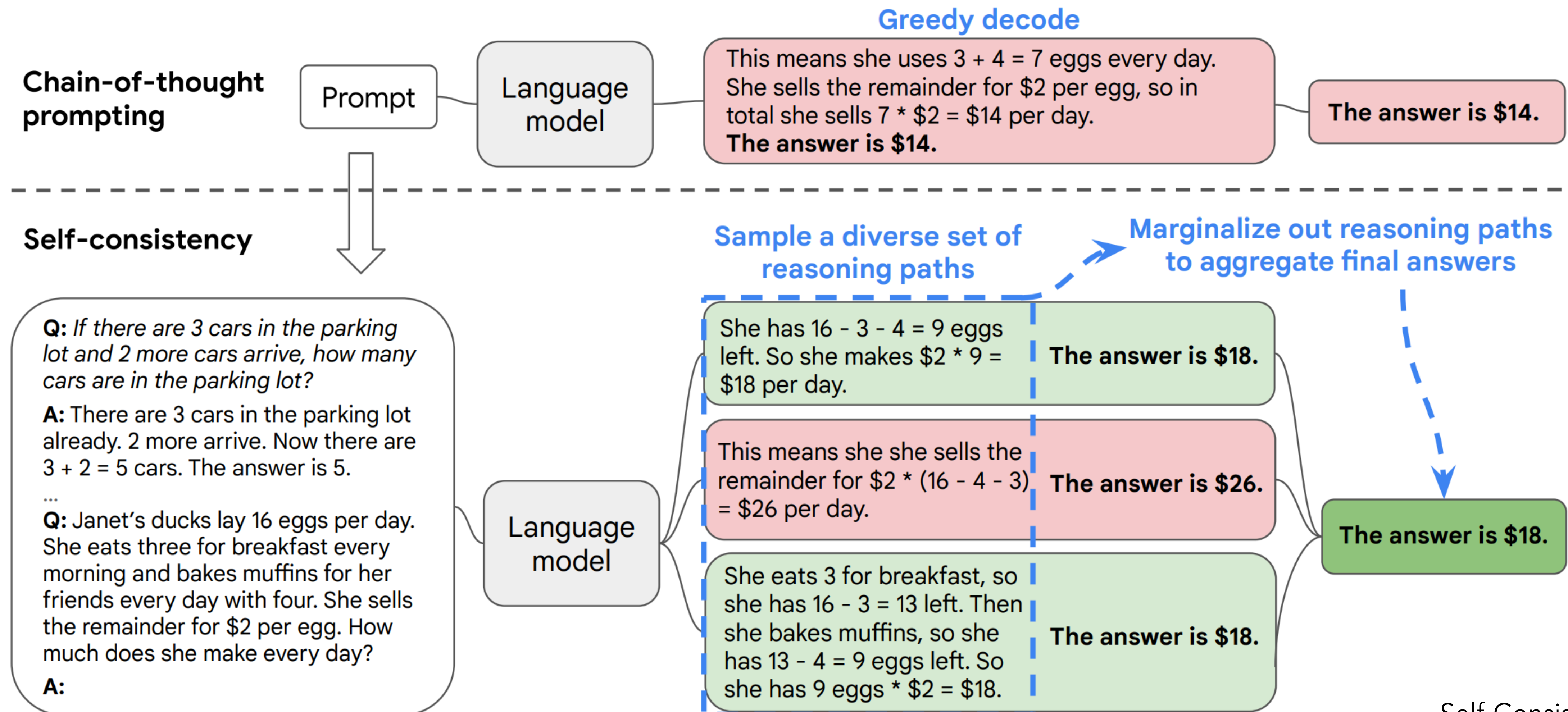
1. **Sample a prompt** x from your dataset.
2. **Sample a completion** using the LLM: $y \sim \pi_{\theta}(\cdot \mid x)$.
3. **Compute reward**: $R := R(x, y)$.
4. **Update**: take a gradient ascent step, weighted by the reward value, i.e. $\theta \leftarrow \theta + \eta R \nabla_{\theta} \log \pi_{\theta}(y \mid x)$

Scaling **reasoning length** as a form of scalable *search*.



As before, **greedy sampling** isn't enough!

Many different latent reasoning paths **z** can lead to equivalent **y**.



Strategies for scaling search *compose*^{*} over CoT!

- Best-of- N or tree search with a scoring function.
- Self-refinement loops with CoT in generation and critique.
- Beyond self-consistency: Joint attention over all N samples.

Lots of ways to scale model reasoning, but many of them can easily be “distilled back” into (longer) chains of thought!

** well, in principle! lots of research was done to generalize these ideas formally so this can be done in practice!*

Language Agents

Suppose we invest heavily in scaling **model size**, **pre-training**, **post-training**, and **reasoning search** for a Transformer.

Are there any fundamental limitations this still cannot handle?

- **Static** — Can't learn about the new or private topics in the world. Can't learn from long-term feedback. Can't *affect* the world!
- **Opaque** — Can't reliably control or attribute decisions or sources!
- **Inefficient** — Loads of esoteric *fact knowledge* use up parameters. *Computations* like arithmetic or algorithms are handled via DNN!

Idea: **Allow the models to invoke tools**!

- Retrieval, e.g. **Web Search** or email search.
- General computation, e.g. calculators or **code execution**.

How can we formulate this paradigm via LLMs?

- Everything is just tokens: Define some special syntax for the LLM (how?), with which it can decide to “invoke” tools.
- Pause the decoding loop, run the tool, and pass back its output.

From models to *Agents* — *what is an agent?*

- A long-studied concept in AI, e.g. in RL.
- Agents are a metaphor or abstraction: *Any* system that can be studied as **working iteratively towards a goal, inside some environment**.
- LLMs increased interest in Agents due to language priors.
- **From tools and observations expressed in language, a strong post-trained LLM can be a powerful agent without [much] training!**

ReAct: Reasoning and Acting with explicit “tools”

(2) AlfWorld

You are in the middle of a room. Looking quickly around you, you see a cabinet 6, a cabinet 1, a coffee machine 1, a countertop 3, a stove burner 1, and a toaster 1.

Your task is to: Put some pepper shaker on a drawer.

(2a) Act-Only

Act 1: Go to drawer 1

Obs 1: The drawer 1 is closed.

Act 2: Open drawer 1

Obs 2: You open the drawer 1. The drawer 1 is open. In it, you see a dish sponge 2, and a spoon 1.

Act 3: Go to sinkbasin 1

Obs 3: On the sinkbasin 1, you see a dish sponge 3, a spatula 1, and a spoon 2.

Act 4: Take peppershaker 1 from sinkbasin 1

Obs 4: Nothing happens.

Act 5: Take peppershaker 1 from sinkbasin 1

Obs 5: Nothing happens.



(2b) ReAct (Reason + Act)

Act 1: **Think**[First I need to find a pepper shaker...more likely to appear in cabinets (1-6), countertops (1-3), ...]

Act 2: Go to cabinet 1

Obs 2: On the cabinet 1, you see a vase 2.

(Here the agent go to cabinet 1, then cabinet 2, 3, then countertop 1 and 2)

Act 6: Go to countertop 3

Obs 6: On the countertop 3, you see a apple 1, a bread 1, a pepper shaker 1, and a vase 3.

Act 7: Take pepper shaker 1 from countertop 3

Obs 7: You pick up the pepper shaker 1 from the countertop 3.

Act 8: **Think**[Now I find a pepper shaker 1. Next, I need to put it in/on drawer 1.]

Act 9: Go to drawer 1

Obs 9: Drawer 1 is closed.

Act 10: Open drawer 1

Obs 10: You open Drawer 1 ...

Act 11: Put pepper shaker 1 in/on drawer 1

Obs 11: You put pepper shaker 1 in/on the drawer 1.



CodeAct: Reasoning and Acting with code execution

Example

User:

① Find the sum of the reciprocals of the roots of $x^2 - 13x + 4 = 0$.

Assistant:

② To find the sum of the reciprocals of the roots of the quadratic equation $x^2 - 13x + 4 = 0$ [...]

<execute>

```
import sympy
x = sympy.Symbol('x')
roots = sympy.solve(x**2 - 13*x + 4)
print(1/roots[0] + 1/roots[1])
```

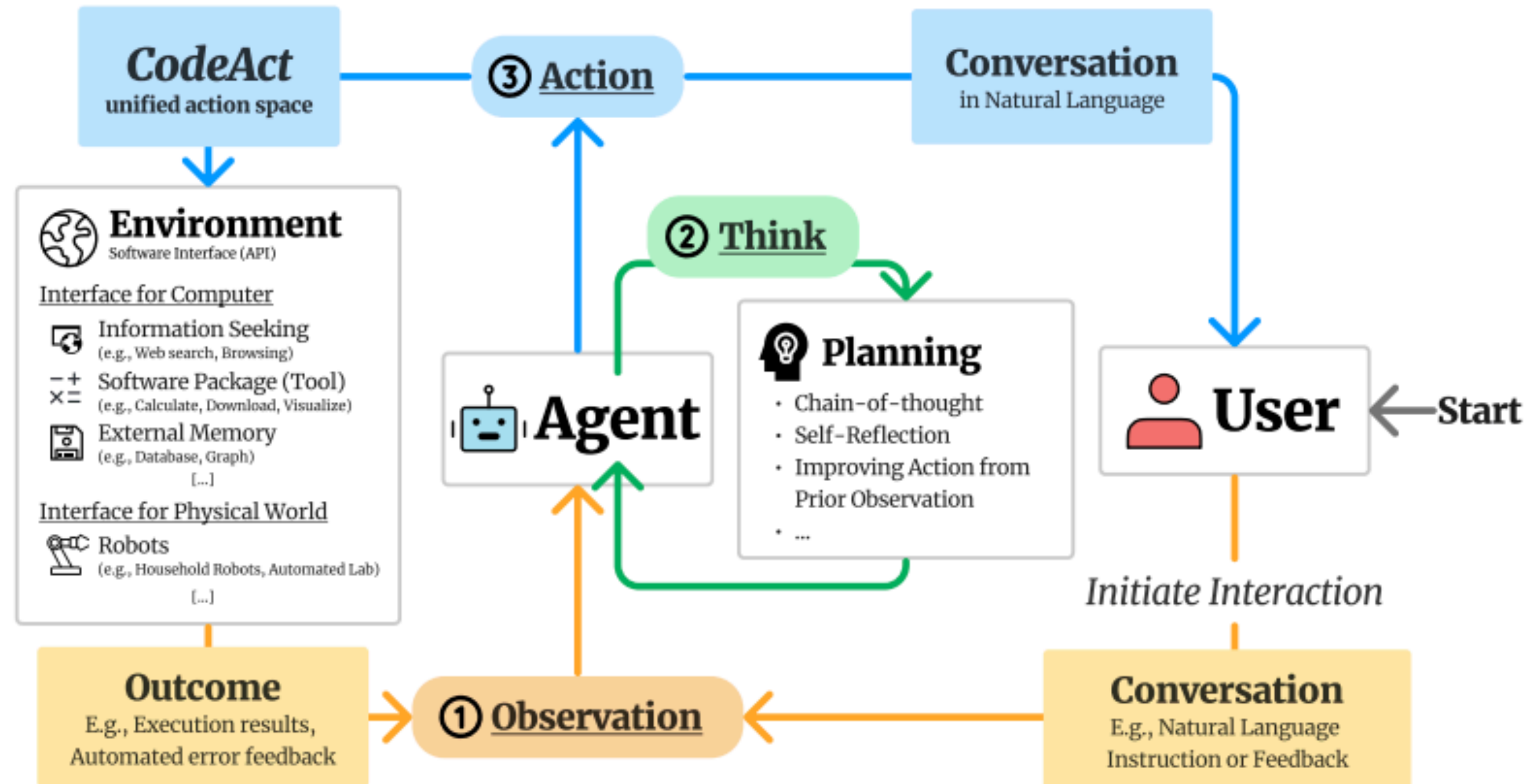
</execute>

Env:

① $1/(3\sqrt{17}/2 + 13/2) + 1/(13/2 - 3\sqrt{17}/2)$

Assistant:

② The roots obtained are in symbolic form, which makes it difficult to evaluate the sum directly. Let's use the quadratic formula to explicitly find the numerical values of the roots first. [...]



Why might this be much more (*and much less!!*) preferable to ReAct?

Scaling post-training for “Agent Models”

- To support strong agents, models need to be able to get invoked in the middle of a long, goal-directed trajectory and...
 - Continue working towards the goal!
 - Recover from their own past errors! A simple language model may repeat it, as the “future often looks like the past”. But an agent model is goal-directed, not just modeling likely sequences!
 - Have an appropriate amount of (dis)trust for the environment!
- Collect many environments, which define initial states (inputs), transitions (e.g., tools), and rewards.
- And use RL, e.g. REINFORCE, as usual.

Recursive Language Models

What if the agent can **write short programs to search for ways to scale its own language modeling** on each difficult problem?

Given arbitrarily long instructions and data...

1. Throw them into a **Python variable**.
2. Put the agent in a **Jupyter notebook, i.e. persistent REPL environment**.
3. In which the LLM is to poke around and **invoke itself on smaller tasks!**

REPL Python Notebook

In[1]

```
# Given the sub-LLM can handle ~0.5M chars and total is ~1.06M, split into two halves
half = len(context)//2
first_half = "\n".join(context[:len(context)//2])
second_half = "\n".join(context[len(context)//2:])

query = """You are given a large corpus of documents
about a trading card game (likely Yu-Gi-Oh!).
I need you to identify ONE specific card that
matches ALL of these clues:
- ...""" + first_half

ans1 = llm_query(query)
print(ans1[:2000])
```

llm_query(...)

Recursive LM
Call
(depth=1)

Out[1]

Best single match: Ally of Justice Catastor

Why it fits (clue → evidence from the provided documents)

– Monster card with printed Level below 8

⋮

In[N]

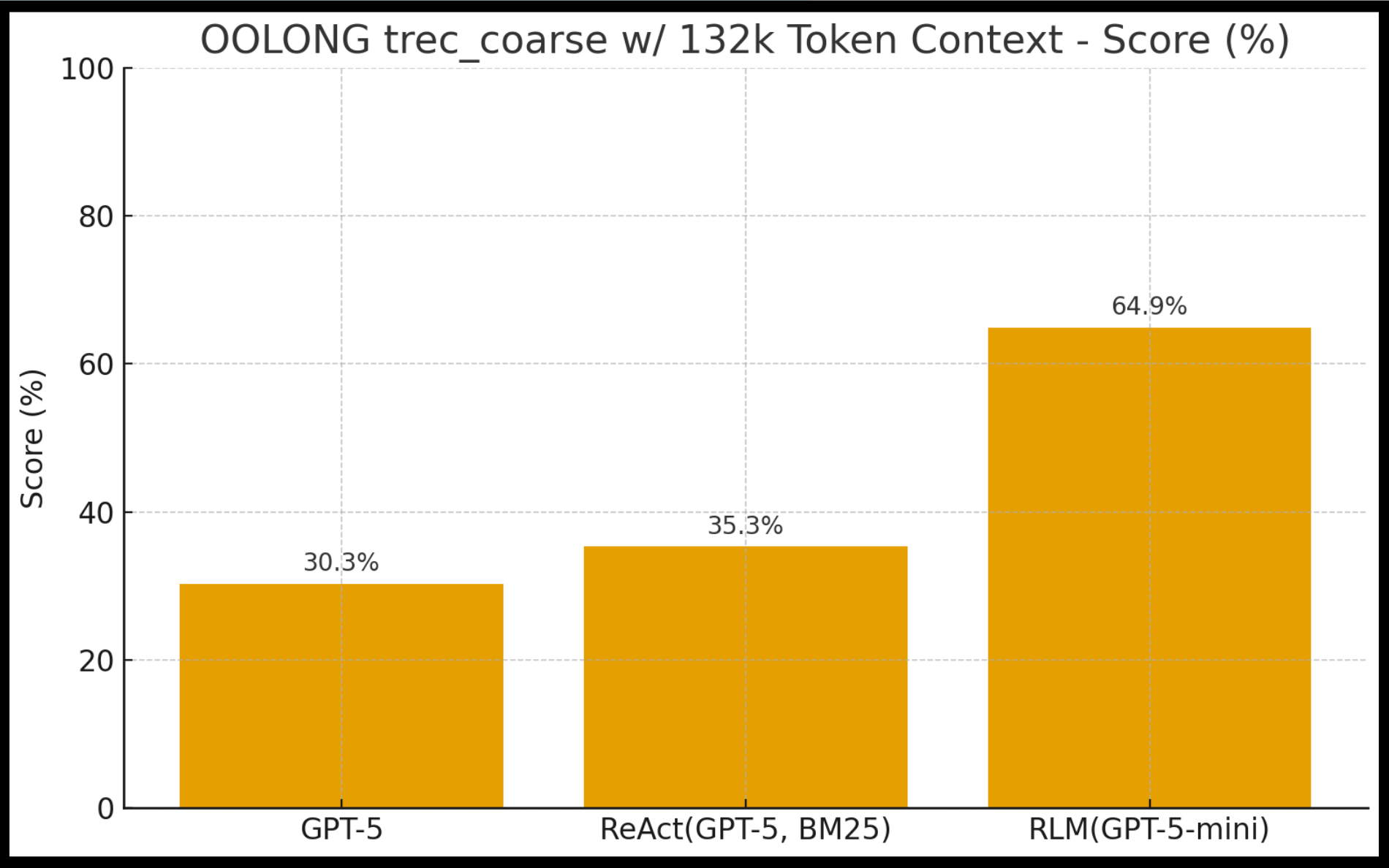
```
# Verify chunk 18 contains the "illegal to play" line and show an excerpt
def find_excerpt(txt, term, window=200):
    i = txt.lower().find(term.lower())
    if i == -1:
        return None
    s = max(0, i-window)
    e = min(len(txt), i+len(term)+window)
    return txt[s:e]

chunk18 = context[18]
illegal_excerpt = find_excerpt(chunk18, "illegal to play")
print("Excerpt from chunk 18 around 'illegal to play':\n", illegal_excerpt or "Not found")
```

Out[N]

Excerpt from chunk 18 around 'illegal to play':

This defeat of "Catastor" appears in the artwork of "Mind Pollutant", where "Fabled Raven" was controlling "Thunder Lord". This monster has an "lswarm" counterpart, "Evilswarm Golem". This card was illegal to play in TCG tournaments following its initial North American release in Duel Terminal – Preview Wave 2. This was because the Duel Terminal Ally of Justice Catastor – (Level 5 Synchro [chunk 20]; released in 2009 with multiple rarities that year: Duel Terminal – Preview Wave 2 (R) and Hidden Arsenal (Se



Wager: Is Attention *all* you need? IsAttentionAllYouNeed.com

On January 1, 2027, a Transformer-like model will continue to hold the state-of-the-art position in most benchmarked tasks in natural language processing.

For the Motion

Jonathan Frankle
@jefrankle
Harvard Professor
Chief Scientist Mosaic ML



Against the Motion

Sasha Rush
@srush_nlp
Cornell Professor
Research Scientist Hugging Face 🤗



Wager

The wager is for donation of equity in Mosaic ML or Hugging Face to a charity of the winner's

Is Attention All You Need?



Current Status: Yes

Time Remaining: 414d 1h 16m 0s

Hmm. With the centrality of explicit reasoning & tool use to the expressive power of a modern LLM, is it really just a Transformer??